



# Yamuna Infra

Admin Portal — Full Build Documentation

React + Vite + JSX · Tailwind CSS · Cloudinary · Express backend

Single source of truth · Module → Phase blueprint

Project folder: `infra-website` · Backend: `infra/server`

Version 1.0 · Generated 2026-06-09

## 0. Read First — How To Build (client instructions)

---

These are the binding instructions for this portal. Everything below expands on them.

1. **Separate web app, outside the mobile app.** A new folder `infra-website` sits beside the React-Native app ( `infra/` ). It is the admin control panel.
2. **Stack: React + Vite + JavaScript (JSX), NOT TypeScript.** Styling with **Tailwind CSS**. Keep it clean and stylish.
3. **One backend only.** Reuse the existing Express server at `infra/server` (same MySQL DB the app uses). Add `/api/admin/*` routes there — do **not** spin up a second backend.
4. **Everything dynamic.** Whatever the app shows — services, food menus, doctors, cars/autos, housekeeping, wheelchair, temples, community, rewards, etc. — must be **created/edited/removed by the admin**. The app reads exactly what the admin writes.
5. **Images via Cloudinary** with a **drag-&-drop uploader** and an **upload-by-link** (paste URL) option, reused everywhere an image is needed.
6. **Full user-management portal** inside the admin (residents, KYC, bookings, status).
7. **Module → Phase structure.** This PDF lists modules; each has phases (large sub-tasks). Build **one module at a time, end-to-end**, then the next.
8. **Track progress in `contextadmin.md`** — every module + the one currently in progress — so work can resume after closing.

**Golden rule:** if the app screen shows it, the admin must be able to manage it. This document maps every app surface to an admin module.

# Table of Contents

| Section                                      | Title                                          | Status     |
|----------------------------------------------|------------------------------------------------|------------|
| 0                                            | Read First — How to build                      | Defined    |
| 1                                            | Purpose, scope & principles                    | Defined    |
| 2                                            | System architecture                            | Defined    |
| 3                                            | Tech stack & conventions                       | Defined    |
| 4                                            | App inventory → what admin manages             | Defined    |
| 5                                            | Data model overview                            | Defined    |
| 6                                            | Authentication, roles & security               | Defined    |
| 7                                            | Media handling (Cloudinary + drag-drop + link) | Defined    |
| 8                                            | Reusable UI component library                  | Defined    |
| PART B — ADMIN MODULES (build one at a time) |                                                |            |
| A0                                           | Foundation & Shell                             | Scaffolded |
| A1                                           | Admin Auth & RBAC                              | Planned    |
| A2                                           | Dashboard & Analytics                          | Planned    |
| A3                                           | Users & Residents                              | Planned    |
| A4                                           | Services & Providers                           | Planned    |
| A5                                           | Food Ordering                                  | Planned    |
| A6                                           | Doctors & Healthcare                           | Planned    |
| A7                                           | Mobility Aids                                  | Planned    |
| A8                                           | Wellness & Spiritual                           | Planned    |
| A9                                           | Temple Directory                               | Planned    |
| A10                                          | Transport (Cabs/Auto/Bus)                      | Planned    |
| A11                                          | Amenities & Clubhouse                          | Planned    |
| A12                                          | Community & Visitors                           | Planned    |
| A13                                          | Rewards & Projects                             | Planned    |
| A14                                          | Notifications & Broadcast                      | Planned    |
| A15                                          | AI Concierge (RAG knowledge base)              | Planned    |
| A16                                          | Payments & Reports                             | Planned    |
| A17                                          | Media Library                                  | Planned    |
| A18                                          | App Settings & Content                         | Planned    |
| A19                                          | Audit Logs & Role Admin                        | Planned    |
| PART C — PROCESS & DELIVERY                  |                                                |            |
| 9                                            | Delivery process & contextadmin.md             | Defined    |

|    |                                     |         |
|----|-------------------------------------|---------|
| 10 | Deployment                          | Defined |
| 11 | Appendix — endpoints, env, glossary | Defined |

# PART A — Foundations

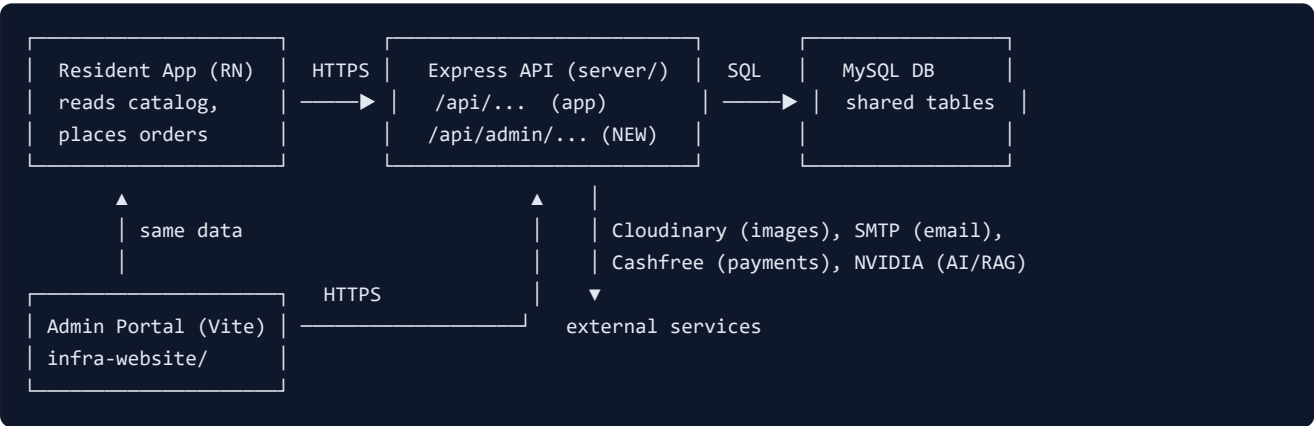
## 1. Purpose, scope & principles

The Yamuna Infra resident mobile app (React Native) today ships with hard-coded / mock content for every service. This portal replaces that with a **fully admin-managed** backend so non-technical staff can run the whole platform: add a new cook, change a dosa's price and photo, add a temple, define a cab fare, broadcast a notice, approve a KYC, etc.

### Core principles

- **Dynamic-first:** every list/card/price/image in the app comes from the database, written by the admin. No app rebuild needed to change content.
- **One backend, one DB:** the portal and the app talk to the same Express server. Admin writes; app reads.
- **CRUD parity:** for each app surface there is a matching admin screen with Create / Read / Update / Delete + enable/disable + ordering.
- **Image-rich:** Cloudinary-backed images on every catalog entity (providers, food, doctors, temples, projects, amenities...).
- **Role-based:** superadmin / manager / support with scoped permissions.
- **Incremental:** ship module by module; never a big-bang.

## 2. System architecture



- **Resident app:** `infra/` (React Native CLI). Already built (Modules 1–36).
- **Backend:** `infra/server/` (Express MVC, mysql2 pool, JWT). Admin routes are added here under `/api/admin` behind an admin-auth middleware.
- **Admin portal:** `infra-website/` (this project). Talks to the backend only; no direct DB access.
- **External:** Cloudinary (media), Cashfree (payments), SMTP/Gmail (email), NVIDIA NIM (LLM/embeddings for AI module).

## 3. Tech stack & conventions

| Layer      | Choice                         | Notes                                                                |
|------------|--------------------------------|----------------------------------------------------------------------|
| Build tool | Vite 5                         | <code>npm run dev</code> on :5173, proxy <code>/api</code> → :4000   |
| UI         | React 18 (JSX)                 | <b>No TypeScript.</b> Function components + hooks only               |
| Styling    | Tailwind CSS 3                 | Brand colors in <code>tailwind.config.js</code> ; no CSS-in-JS       |
| Routing    | react-router-dom 6             | Nested routes per module; protected by auth guard                    |
| HTTP       | axios                          | Single instance <code>src/lib/api.js</code> + JWT interceptor        |
| State      | Local + React Query (optional) | Keep it simple; lift only what's shared                              |
| Images     | Cloudinary                     | Unsigned preset; drag-drop + URL; <code>src/lib/cloudinary.js</code> |

|         |                    |                                                                      |
|---------|--------------------|----------------------------------------------------------------------|
| Backend | Express (existing) | Controllers/models/routes pattern already in <code>server/src</code> |
| Auth    | JWT (admins table) | Separate from resident OTP login; role claim                         |

### Frontend folder layout (target)

```
infra-website/
  src/
    lib/      api.js, cloundinary.js, auth.js, format.js
    components/ DataTable.jsx, FormModal.jsx, ImageUploader.jsx,
               ConfirmDialog.jsx, Toast.jsx, Pagination.jsx, SearchBar.jsx,
               StatusBadge.jsx, PageHeader.jsx, Sidebar.jsx, Topbar.jsx
    layouts/   AdminLayout.jsx, AuthLayout.jsx
    pages/
      auth/    Login.jsx
      dashboard/ Dashboard.jsx
      users/    UsersList.jsx, UserDetail.jsx
      services/ Categories.jsx, Providers.jsx, Offerings.jsx
      food/     Categories.jsx, Items.jsx, Orders.jsx
      ... (one folder per module) ...
    App.jsx, main.jsx, index.css
```

### Backend additions (in `server/src` )

```
middleware/  requireAdmin.js (JWT + role check)
routes/      adminRoutes.js → mounts /api/admin/* sub-routers
controllers/ admin*Controller.js (thin)
models/      reuse existing models + new Admin*, generic CRUD helpers
migrations/  0XX_admin_*.sql (admins, audit_logs, + columns like image_url)
```

## 4. App inventory → what the admin manages

Every row below is a thing the app shows today. The right column is the admin module that will own it.

| App surface                         | Entities (dynamic)                                                                                | Admin module |
|-------------------------------------|---------------------------------------------------------------------------------------------------|--------------|
| Home dashboard                      | Quick tiles, banners, KPIs                                                                        | A2 / A18     |
| Home Cleaning / Housekeeping / Cook | Service categories, providers, <b>offerings</b> (price/unit/image)                                | A4           |
| Meal / Food                         | Food categories (icon/image), food items (image, price, veg, rating), orders                      | A5           |
| Doctor & Healthcare                 | Specialties, doctors (photo, fee, rating), slots, appointments, medicine orders                   | A6           |
| Mobility Aid                        | Equipment (wheelchair, etc.), pricing, requests                                                   | A7           |
| Wellness & Spa                      | Therapies/services (image, price, duration), bookings                                             | A8           |
| Spiritual Concierge                 | Spiritual services, pujas, bookings                                                               | A8           |
| Temple Directory                    | Temples (image, aarti times, crowd, distance, maps/donation URL, VIP)                             | A9           |
| Darshan & Transport                 | Vehicle types + fares, Vrindavan places (coords), rides                                           | A10          |
| Community — Feed                    | Announcements (pin/category), events                                                              | A12          |
| Community — Visitors                | Visitor passes, approvals, gate logs                                                              | A12          |
| Community — Amenities               | Amenity categories, facilities (capacity/deposit/rate/features/image), slots, blackouts, bookings | A11          |

|                           |                                                                |           |
|---------------------------|----------------------------------------------------------------|-----------|
| Rewards & Referrals       | Reward offers (points, partner, image), redemptions, referrals | A13       |
| New Projects / Investment | Projects (image, price, status, description)                   | A13       |
| Vrindavan Companion (AI)  | Knowledge base (PDF/DB/docs/instructions), daily content       | A15 / A18 |
| Reminders                 | Reminder categories                                            | A18       |
| Notifications             | Broadcasts, per-type templates                                 | A14       |
| Profile / KYC / Family    | Resident profiles, KYC review                                  | A3        |
| SOS                       | SOS requests, emergency contacts, dispatch logs                | A3 / A12  |
| Payments                  | Transactions, refunds, ledgers                                 | A16       |

## 5. Data model overview

Most catalog tables follow a common shape so a *generic CRUD* backend + a *generic DataTable* frontend can serve many modules. Add an `image_url` column (Cloudinary) and `is_active`, `sort_order` to every catalog table.

### Common columns (every catalog entity)

| Column                  | Type      | Purpose               |
|-------------------------|-----------|-----------------------|
| id                      | BIGINT PK | Identity              |
| name / title            | VARCHAR   | Display               |
| description             | TEXT      | Details               |
| image_url               | VARCHAR   | Cloudinary secure_url |
| price / amount          | INT       | Where applicable (₹)  |
| is_active               | TINYINT   | Show/hide in app      |
| sort_order              | INT       | Display order         |
| created_at / updated_at | DATETIME  | Audit                 |

### Key entity groups

- **Catalog:** service\_categories, service\_providers, provider\_offerings, food\_categories, food\_items, doctors, specialties, mobility\_equipment, wellness\_services, spiritual\_services, temples, vehicle\_types, transport\_places, amenity\_categories, amenities, reward\_offers, projects.
- **Transactional:** bookings (per service), food\_orders, appointments, rides, amenity\_bookings, visitor\_passes, sos\_requests, payments, redemptions, referrals, reminders.
- **People:** users (residents), family\_members, kyc, emergency\_contacts, admins, roles.
- **Content/Config:** announcements, events, daily\_content, notifications, app\_settings, reminder\_categories, ai\_knowledge\_sources.

Several of these tables already exist for the app (Modules 1–36). The admin work mostly adds **write** endpoints + **image\_url/is\_active/sort\_order** columns + admin UIs on top of existing read endpoints.

## 6. Authentication, roles & security

- **Admins are separate from residents.** New `admins` table (email, password\_hash, role, name, is\_active). Resident OTP login is untouched.
- **Login:** email + password → bcrypt verify → JWT ( { sub, role, type:'admin' } , 12h). Optional OTP/2FA later.
- **Roles:** `superadmin` (everything incl. admin/role mgmt), `manager` (all content + users + orders), `support` (read + limited actions: KYC review, order status, notifications).
- **Middleware:** `requireAdmin(roles?)` verifies the JWT, checks `type==='admin'` and role membership.
- **Frontend guard:** a `<Protected/>` wrapper redirects to `/login` when no token; hides nav items the role can't access.
- **Security:** rate-limit login, helmet, CORS allowlist (portal origin), validate every body with Joi (already used), never expose service keys to the browser (Cashfree/NVIDIA stay server-side; only Cloudinary unsigned preset is public).

## 7. Media handling — Cloudinary

- **Component:** `<ImageUploader value onChange />` used by every image field. Two ways to set an image:
  1. **Drag & drop** (or click to pick) a file → uploads to Cloudinary → stores `secure_url`.
  2. **Paste a link** → validates & stores the URL (optionally re-hosted via Cloudinary fetch).



- **Unsigned preset** for browser uploads ( `VITE_CLOUDINARY_CLOUD_NAME` + `VITE_CLOUDINARY_UPLOAD_PRESET` ). For stricter control add a backend `/api/admin/media/sign` route (signed uploads).
- **Media Library (A17)**: browse/search past uploads, organise in folders, reuse via a picker.
- **Transforms**: store base `secure_url` ; the app/portal request sized variants via Cloudinary URL params (e.g. `w_400,q_auto,f_auto` ).

## 8. Reusable UI component library

Build these once (Module A0) and reuse across every module — this is what makes 20 modules feasible.

| Component                                         | Responsibility                                                                          |
|---------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>AdminLayout</code>                          | Sidebar + topbar + content outlet                                                       |
| <code>PageHeader</code>                           | Title, subtitle, primary action button (e.g. "+ Add")                                   |
| <code>DataTable</code>                            | Columns, rows, sort, pagination, row actions (edit/delete/toggle), empty/loading states |
| <code>SearchBar</code> / <code>FilterChips</code> | Debounced search + status/category filters                                              |
| <code>FormModal</code>                            | Slide-over/modal form; fields config-driven; validation; submit                         |
| <code>ImageUploader</code>                        | Drag-drop + URL paste → Cloudinary → URL                                                |
| <code>ConfirmDialog</code>                        | Destructive-action confirmation                                                         |
| <code>StatusBadge</code>                          | Colored status pills (active, pending, paid...)                                         |
| <code>Toast</code>                                | Success/error notifications                                                             |
| <code>StatCard</code>                             | Dashboard KPI tile                                                                      |

**Pattern:** a typical catalog module = `PageHeader` + `SearchBar` + `DataTable` + `FormModal` (with `ImageUploader` ) + `ConfirmDialog` . ~150 lines per module once the library exists.

## PART B — Admin Modules

---

Each module below is built **one at a time, end-to-end** (backend migration → endpoints → frontend screens → test → mark done in `contextadmin.md`). Each module lists: **Goal**, **Manages**, **Phases** (sub-tasks), **Backend**, **Frontend**, **Acceptance**.

## A0 — Foundation & Shell Scaffolded

**Goal:** the project skeleton + reusable pieces everything else depends on.

**Manages:** nothing yet — infrastructure.

### Phases

| #   | Phase                                            | Done when                                                    |
|-----|--------------------------------------------------|--------------------------------------------------------------|
| 0.1 | Vite + React + JSX + Tailwind scaffold           | <code>npm run dev</code> serves a styled shell ✓ (done)      |
| 0.2 | AdminLayout (sidebar + topbar) + routing         | All module routes render placeholders ✓ (done)               |
| 0.3 | API client + error/toast handling                | <code>api.js</code> with JWT interceptor, global error toast |
| 0.4 | Reusable components (DataTable, FormModal, etc.) | Library in <code>src/components</code> documented            |
| 0.5 | ImageUploader (drag-drop + URL → Cloudinary)     | Returns a URL; used by a demo field                          |
| 0.6 | Theme + responsive layout polish                 | Looks good on laptop + tablet widths                         |

### Acceptance

- Shell runs, navigation works, components reusable, image upload returns a Cloudinary URL.

## A1 — Admin Auth & RBAC Planned

**Goal:** secure admin login with roles; protect everything.

**Manages:** admin accounts, sessions, roles.

### Phases

| #   | Phase                            | Sub-tasks                                                                                               |
|-----|----------------------------------|---------------------------------------------------------------------------------------------------------|
| 1.1 | Backend admins + auth            | migration <code>admins</code> ; seed superadmin; <code>POST /api/admin/auth/login</code> (bcrypt + JWT) |
| 1.2 | requireAdmin middleware + RBAC   | verify JWT, role guard, <code>GET /api/admin/auth/me</code>                                             |
| 1.3 | Login page                       | email/password form, error states, token storage                                                        |
| 1.4 | Route guards + role-aware nav    | <code>&lt;Protected/&gt;</code> , hide unauthorized nav                                                 |
| 1.5 | Profile, logout, change password | topbar menu, <code>POST /auth/change-password</code>                                                    |

### Backend

```
POST /api/admin/auth/login      { email, password } → { token, admin }
GET  /api/admin/auth/me        → { id, name, role }
POST /api/admin/auth/change-password
migration: admins(id, name, email, password_hash, role, is_active, created_at)
```

### Acceptance

- Only valid admins can log in; role restricts nav + endpoints; token expiry handled.

## A2 — Dashboard & Analytics Planned

**Goal:** at-a-glance health of the platform.

**Manages:** read-only aggregates.

### Phases

| #   | Phase             | Sub-tasks                                                           |
|-----|-------------------|---------------------------------------------------------------------|
| 2.1 | Stats endpoints   | counts: residents, active services, orders today, revenue, open SOS |
| 2.2 | KPI cards         | StatCard grid bound to stats                                        |
| 2.3 | Charts            | orders/revenue over time (lightweight chart lib or SVG)             |
| 2.4 | Recent activity   | latest orders / bookings / signups feed                             |
| 2.5 | Date-range filter | today / 7d / 30d toggle                                             |

### Backend

```
GET /api/admin/stats/overview?range=7d
GET /api/admin/stats/timeseries?metric=orders&range=30d
GET /api/admin/activity/recent
```

### Acceptance

- Dashboard loads real numbers; charts render; range filter works.

## A3 — Users & Residents Planned

**Goal:** manage residents — the user-management portal.

**Manages:** users, family\_members, kyc, emergency\_contacts, their bookings.

### Phases

| #   | Phase                   | Sub-tasks                                                            |
|-----|-------------------------|----------------------------------------------------------------------|
| 3.1 | Residents list + search | DataTable: name, mobile, unit, KYC status; search/filter; pagination |
| 3.2 | Resident detail         | personal, family, preferences, emergency contacts                    |
| 3.3 | KYC review workflow     | view ID, approve/reject with reason → updates app status             |
| 3.4 | Bookings per user       | all orders/rides/appointments for a resident                         |
| 3.5 | Status actions          | block/unblock, reset, notes                                          |

### Backend

```
GET /api/admin/users?search=&kyc=&page=
GET /api/admin/users/:id (profile + family + kyc + contacts)
POST /api/admin/users/:id/kyc/approve | /reject
POST /api/admin/users/:id/status { active }
GET /api/admin/users/:id/bookings
```

### Acceptance

- Find any resident, review KYC, see their activity, change status.

## A4 — Services & Providers Planned

**Goal:** manage Home Cleaning / Housekeeping / Cook and any future service: categories → providers → offerings.

**Manages:** service\_categories, service\_providers, provider\_offerings.

### Phases

| #   | Phase                         | Sub-tasks                                                                    |
|-----|-------------------------------|------------------------------------------------------------------------------|
| 4.1 | Service categories CRUD       | name, icon, image, is_active, sort_order                                     |
| 4.2 | Providers CRUD                | name, tagline, image, phone, gender, rating, experience, priceFrom, category |
| 4.3 | Offerings CRUD (per provider) | name, description, price, unit, image, is_active                             |
| 4.4 | Availability & ordering       | enable/disable, drag-to-reorder, featured                                    |
| 4.5 | Bookings view                 | incoming service bookings + status updates                                   |

### Backend (CRUD pattern repeated per entity)

```
GET|POST      /api/admin/services/categories
PUT|DELETE    /api/admin/services/categories/:id
GET|POST      /api/admin/services/providers
PUT|DELETE    /api/admin/services/providers/:id
GET|POST      /api/admin/services/providers/:id/offerings
PUT|DELETE    /api/admin/services/offerings/:id
GET           /api/admin/services/bookings
```

### Frontend

- Categories page → click → Providers page → click → Offerings page; each with DataTable + FormModal + ImageUploader.

### Acceptance

- Admin adds "FreshNest Cleaners" with offerings + images; it appears instantly in the app's drill-down.

## A5 — Food Ordering Planned

**Goal:** run the in-app food app — categories, menu items, orders.

**Manages:** food\_categories, food\_items, food\_orders.

### Phases

| #   | Phase                | Sub-tasks                                                                                                        |
|-----|----------------------|------------------------------------------------------------------------------------------------------------------|
| 5.1 | Food categories CRUD | name, icon, cover image, sort_order (Breakfast, Lunch Thali, Dinner, South Indian, Snacks, Sweets, Beverages...) |
| 5.2 | Food items CRUD      | name, description, <b>image</b> , price, veg/non-veg, rating, category, is_active                                |
| 5.3 | Stock / availability | mark sold-out, daily availability                                                                                |
| 5.4 | Orders management    | list orders, update status (preparing → out → delivered)                                                         |
| 5.5 | Tiffin plans         | manage monthly subscription plans                                                                                |

### Backend

```
GET|POST  /api/admin/food/categories  PUT|DELETE /:id
GET|POST  /api/admin/food/items       PUT|DELETE /:id
GET       /api/admin/food/orders      PUT  /api/admin/food/orders/:id/status
```

### Acceptance

- Add "Masala Dosa" with photo & price under South Indian → shows in app menu; orders appear here with status control.



## A6 — Doctors & Healthcare Planned

**Goal:** manage doctors, specialties, slots, appointments, medicine orders.

**Manages:** specialties, doctors, doctor\_slots, appointments, medicine\_orders.

### Phases

| #   | Phase            | Sub-tasks                                                                  |
|-----|------------------|----------------------------------------------------------------------------|
| 6.1 | Specialties CRUD | name, icon                                                                 |
| 6.2 | Doctors CRUD     | name, photo, specialty, qualifications, fee, rating, experience, is_active |
| 6.3 | Slot templates   | weekly availability / day slots per doctor                                 |
| 6.4 | Appointments     | list, confirm/cancel, reschedule                                           |
| 6.5 | Medicine orders  | incoming orders, fulfil/track                                              |

### Backend

```
GET|POST /api/admin/healthcare/specialties PUT|DELETE /:id
GET|POST /api/admin/healthcare/doctors PUT|DELETE /:id
PUT      /api/admin/healthcare/doctors/:id/slots
GET      /api/admin/healthcare/appointments PUT /:id/status
GET      /api/admin/healthcare/medicine-orders
```

### Acceptance

- Add a cardiologist with photo, fee, slots → bookable in app; appointments manageable here.

## A7 — Mobility Aids Planned

**Goal:** manage wheelchair & mobility equipment + requests.

**Manages:** mobility\_equipment, mobility\_requests.

### Phases

| #   | Phase                  | Sub-tasks                                            |
|-----|------------------------|------------------------------------------------------|
| 7.1 | Equipment catalog CRUD | name, image, type, rent/price, deposit, availability |
| 7.2 | Pricing & durations    | hourly/daily/weekly rates                            |
| 7.3 | Requests management    | list, assign, mark delivered/returned                |

### Backend

```
GET|POST /api/admin/mobility/equipment  PUT|DELETE /:id
GET      /api/admin/mobility/requests   PUT /:id/status
```

### Acceptance

- Add "Electric wheelchair" with photo & daily rate → available in app; requests tracked here.

## A8 — Wellness & Spiritual Planned

**Goal:** manage spa/wellness therapies and spiritual concierge services.

**Manages:** wellness\_services, spiritual\_services, their bookings.

### Phases

| #   | Phase                   | Sub-tasks                                           |
|-----|-------------------------|-----------------------------------------------------|
| 8.1 | Wellness services CRUD  | name, image, description, price, duration, category |
| 8.2 | Spiritual services CRUD | puja/seva types, image, price, notes                |
| 8.3 | Bookings                | list + status for both                              |

### Backend

```
GET|POST /api/admin/wellness/services    PUT|DELETE /:id
GET|POST /api/admin/spiritual/services   PUT|DELETE /:id
GET      /api/admin/wellness/bookings
```

### Acceptance

- Add an "Abhyanga massage" & a "Satyanarayan Puja" with images/prices → bookable in app.

## A9 — Temple Directory Planned

**Goal:** manage the Vrindavan temple directory + darshan config.

**Manages:** temples, darshan\_slots.

### Phases

| #   | Phase                  | Sub-tasks                                                                                                           |
|-----|------------------------|---------------------------------------------------------------------------------------------------------------------|
| 9.1 | Temples CRUD           | name, city, image, rating, crowd status, distanceKm, aarti times, maps URL, donation URL, vipAvailable, description |
| 9.2 | Aarti / timings editor | structured timings per temple                                                                                       |
| 9.3 | Darshan config         | VIP options, slots, fees                                                                                            |
| 9.4 | Featured / ordering    | pin popular temples                                                                                                 |

### Backend

```
GET|POST /api/admin/temples      PUT|DELETE /:id
GET      /api/admin/darshan/bookings
```

### Acceptance

- Add a temple with photo, aarti times, maps link → shows in directory; darshan bookable.

## A10 — Transport (Cabs / Auto / Bus) Planned

**Goal:** run the Ola/Uber-style module — vehicle types, fares, places, rides.

**Manages:** vehicle\_types, transport\_places, fare\_rules, rides.

### Phases

| #    | Phase                      | Sub-tasks                                                      |
|------|----------------------------|----------------------------------------------------------------|
| 10.1 | Vehicle types CRUD         | label, icon/image, capacity, base fare, per-km, ETA, is_active |
| 10.2 | Places management          | Vrindavan/Mathura places with coords (map picker); temple flag |
| 10.3 | Fare rules                 | surge, min fare, night charge, free-km                         |
| 10.4 | Rides view                 | incoming rides, assign/confirm, status                         |
| 10.5 | (Future) Driver management | drivers, assignment, live status                               |

### Backend

```
GET|POST /api/admin/transport/vehicles  PUT|DELETE /:id
GET|POST /api/admin/transport/places    PUT|DELETE /:id
PUT      /api/admin/transport/fare-rules
GET      /api/admin/transport/rides     PUT /:id/status
```

### Acceptance

- Admin edits sedan per-km fare & adds a new place → app fares update; rides land here. Bookings come to us (no driver app yet).

## A11 — Amenities & Clubhouse Planned

**Goal:** manage clubhouse facilities, slots, blackouts, bookings.

**Manages:** amenity\_categories, amenities, amenity\_slots, blackouts, amenity\_bookings.

### Phases

| #    | Phase              | Sub-tasks                                                                  |
|------|--------------------|----------------------------------------------------------------------------|
| 11.1 | Categories CRUD    | Halls, Yoga/Fitness, Pools, Courts, Lawns...                               |
| 11.2 | Facilities CRUD    | name, image, capacity, deposit, hourly rate, location, features, is_active |
| 11.3 | Slot configuration | 2-hour slots, opening hours                                                |
| 11.4 | Blackout calendar  | maintenance dates per facility                                             |
| 11.5 | Bookings           | view + approve/cancel                                                      |

### Backend

```
GET|POST /api/admin/amenities/categories  PUT|DELETE /:id
GET|POST /api/admin/amenities             PUT|DELETE /:id
PUT      /api/admin/amenities/:id/slots
POST     /api/admin/amenities/:id/blackouts
GET      /api/admin/amenities/bookings
```

### Acceptance

- Add "Riverside Lawn" with deposit + features + image; set a blackout date → reflected in app.

## A12 — Community & Visitors Planned

**Goal:** run the community feed + visitor management.

**Manages:** announcements, events, visitor\_passes.

### Phases

| #    | Phase              | Sub-tasks                                             |
|------|--------------------|-------------------------------------------------------|
| 12.1 | Announcements CRUD | title, body, category, pinned, image, publish/expire  |
| 12.2 | Events CRUD        | title, date, location, description, image             |
| 12.3 | Visitor passes     | view, approve/deny, gate logs                         |
| 12.4 | Broadcast          | push an announcement as a notification (links to A14) |

### Backend

```
GET|POST /api/admin/community/announcements  PUT|DELETE /:id
GET|POST /api/admin/community/events        PUT|DELETE /:id
GET      /api/admin/visitors                 PUT /:id/status
```

### Acceptance

- Post a pinned announcement with image → top of app feed; manage visitor approvals.

## A13 — Rewards & Projects Planned

**Goal:** manage reward offers, redemptions, referrals, and new-project listings.

**Manages:** reward\_offers, redemptions, referrals, projects.

### Phases

| #    | Phase                    | Sub-tasks                                               |
|------|--------------------------|---------------------------------------------------------|
| 13.1 | Reward offers CRUD       | title, partner, image, points cost, category, is_active |
| 13.2 | Redemptions              | list, mark fulfilled                                    |
| 13.3 | Projects/Investment CRUD | name, location, image, priceFrom, status, description   |
| 13.4 | Referrals / leads        | view referral signups & investment leads                |

### Backend

```
GET|POST /api/admin/rewards/offers    PUT|DELETE /:id
GET      /api/admin/rewards/redemptions
GET|POST /api/admin/projects          PUT|DELETE /:id
```

### Acceptance

- Add an offer & a new project with images → appear in app; redemptions/leads visible.



## A14 — Notifications & Broadcast Planned

**Goal:** compose and send notifications to residents.

**Manages:** notifications, notification\_targets, templates.

### Phases

| #    | Phase                   | Sub-tasks                                       |
|------|-------------------------|-------------------------------------------------|
| 14.1 | Compose & send (in-app) | title, body, type icon, deep-link target        |
| 14.2 | Targeting               | all / segment (tower, KYC status) / single user |
| 14.3 | Scheduling              | send now or schedule                            |
| 14.4 | History & read stats    | sent list, delivered/read counts                |
| 14.5 | (Future) Push via FCM   | device tokens + FCM send                        |

### Backend

```
POST /api/admin/notifications    { title, body, type, target, schedule }
GET  /api/admin/notifications    (history)
GET  /api/notifications          (app reads its feed)
```

### Acceptance

- Send "Water maintenance" to Tower 2 → appears in residents' Notifications feed.

## A15 — AI Concierge (RAG knowledge base) Planned · big

**Goal:** the configurable brain for the Vrindavan Companion chatbot (app Module 28). Admin controls what the bot knows.

**Manages:** ai\_knowledge\_sources (PDFs, docs, DB connections, instruction snippets), embeddings, vector store.

### Phases

| #    | Phase                       | Sub-tasks                                                                   |
|------|-----------------------------|-----------------------------------------------------------------------------|
| 15.1 | Knowledge sources CRUD      | upload PDF/doc, add URL, add instruction text, connect a DB view            |
| 15.2 | Ingestion + chunking        | extract text, chunk, store                                                  |
| 15.3 | Embeddings + vector store   | NVIDIA <code>nv-embedqa-e5-v5</code> → vector table / store                 |
| 15.4 | Retrieval wiring            | backend <code>/api/ai/chat</code> retrieves context → LLM (key server-side) |
| 15.5 | Test console                | admin chats against the KB; sees sources used                               |
| 15.6 | Per-community KB + fallback | scoping; rule-based fallback when empty                                     |

**Security:** the NVIDIA key must move server-side here (today the app calls NVIDIA directly as an interim). This module completes the secure backend RAG design noted in the app's CONTEXT.md (Module 28).

### Backend

```
POST /api/admin/ai/sources      (upload/add)  GET /api/admin/ai/sources  DELETE /:id
POST /api/admin/ai/reindex      (rebuild embeddings)
POST /api/ai/chat               (app → retrieves context → NVIDIA)  [server-side key]
```

### Acceptance

- Upload a society rulebook PDF → bot answers rule questions from it; admin sees which chunks were used.

## A16 — Payments & Reports Planned

**Goal:** oversee all money — Cashfree transactions, refunds, reports.

**Manages:** payments, refunds, ledgers (read + limited actions).

### Phases

| #    | Phase             | Sub-tasks                                  |
|------|-------------------|--------------------------------------------|
| 16.1 | Transactions list | filters: status, method, date, resident    |
| 16.2 | Refunds           | initiate refund via Cashfree (server-side) |
| 16.3 | Reconciliation    | match webhooks vs orders                   |
| 16.4 | Reports / export  | CSV export, revenue by service             |

### Backend

```
GET  /api/admin/payments?status=&from=&to=
POST /api/admin/payments/:id/refund
GET  /api/admin/payments/export.csv
```

### Acceptance

- See every transaction, filter, refund, export a monthly report.

## A17 — Media Library Planned

**Goal:** central place for all Cloudinary images; reuse picker.

**Manages:** media assets (Cloudinary), folders.

### Phases

| #    | Phase                      | Sub-tasks                                  |
|------|----------------------------|--------------------------------------------|
| 17.1 | Uploader (drag-drop + URL) | finalise the reusable ImageUploader        |
| 17.2 | Library grid               | browse/search uploaded assets              |
| 17.3 | Folders / tags             | organise by module                         |
| 17.4 | Picker integration         | "choose from library" in every image field |

### Acceptance

- Upload once, reuse anywhere; pick existing images in any form.

## A18 — App Settings & Content Planned

**Goal:** manage global app content + config.

**Manages:** daily\_content, app\_settings, reminder\_categories, content pages.

### Phases

| #    | Phase               | Sub-tasks                                   |
|------|---------------------|---------------------------------------------|
| 18.1 | Daily content       | quotes, bhajans, temple suggestion rotation |
| 18.2 | Feature flags       | toggle modules/sections on/off in app       |
| 18.3 | Reminder categories | manage the reminder category list           |
| 18.4 | Content pages       | Terms, Privacy, About (served to app)       |

### Backend

```
GET|PUT /api/admin/settings
GET|POST /api/admin/daily-content  PUT|DELETE /:id
```

### Acceptance

- Change today's quote / toggle a feature / edit Terms → reflected in app.

## A19 — Audit Logs & Role Admin Planned

**Goal:** accountability + manage admin team.

**Manages:** audit\_logs, admins, roles.

### Phases

| #    | Phase                    | Sub-tasks                            |
|------|--------------------------|--------------------------------------|
| 19.1 | Audit logging middleware | record who/what/when on every write  |
| 19.2 | Log viewer               | filter by admin/entity/date          |
| 19.3 | Admin & role management  | invite admins, set roles, deactivate |

### Backend

```
GET /api/admin/audit?admin=&entity=&from=
GET|POST /api/admin/admins PUT /:id (superadmin only)
```

### Acceptance

- Every change is traceable; superadmin manages the team.

# PART C — Process & Delivery

## 9. Delivery process & contextadmin.md

We build **one module at a time, end-to-end**, exactly like the app was built. For each module:

1. Backend migration (new tables / columns like `image_url` ).
2. Backend model + controller + routes under `/api/admin` (+ Joi validation).
3. Quick backend smoke test (curl / REST client).
4. Frontend pages + wire to API using the reusable components.
5. Manual test against the running app (admin writes → app shows).
6. Mark the module/phase done in `contextadmin.md` .

`contextadmin.md` lives in `infra-website/` and mirrors the app's `CONTEXT.md` : a module-status table + a "current focus" section + per-module notes. Always update it at the end of a work session so the next session resumes cleanly.

### Suggested build order

A0 (done) → A1 (auth) → A0.3–0.5 (components + uploader) → A3 (users) → A4 (services) → A5 (food) → A9 (temples) → A10 (transport) → A11 (amenities) → A6 (healthcare) → A8 (wellness/spiritual) → A7 (mobility) → A12 (community) → A13 (rewards/projects) → A14 (notifications) → A2 (dashboard) → A16 (payments) → A17 (media) → A18 (settings) → A15 (AI/RAG) → A19 (audit).

## 10. Deployment

- **Backend:** already on the shared host; add admin routes + run new migrations. Set Cloudinary + admin secrets in `server/.env` .
- **Admin portal:** `npm run build` → static `dist/` . Host on Netlify/Vercel/any static host, or serve via the Express server ( `express.static` ). Lock CORS to the portal origin.
- **Env:** `VITE_API_BASE_URL` (prod API), `VITE_CLOUDINARY_*` . Never put server secrets in the Vite build.
- **Auth:** HTTPS only; short-lived admin JWT; optional IP allowlist for the admin origin.

## 11. Appendix

### 11.1 Endpoint namespace summary

```
/api/admin/auth/*      login, me, change-password
/api/admin/stats/*     dashboard aggregates
/api/admin/users/*     residents, kyc, status, bookings
/api/admin/services/*  categories, providers, offerings, bookings
/api/admin/food/*      categories, items, orders
/api/admin/healthcare/* specialties, doctors, slots, appointments
/api/admin/mobility/*  equipment, requests
/api/admin/wellness/*  services, bookings
/api/admin/spiritual/* services
/api/admin/temples/*   temples; /api/admin/darshan/*
/api/admin/transport/* vehicles, places, fare-rules, rides
/api/admin/amenities/* categories, amenities, slots, blackouts, bookings
/api/admin/community/* announcements, events; /api/admin/visitors
/api/admin/rewards/*   offers, redemptions; /api/admin/projects
/api/admin/notifications compose/send/history
/api/admin/ai/*        sources, reindex
/api/admin/payments/*  list, refund, export
/api/admin/settings, /daily-content
/api/admin/audit, /api/admin/admins
```

### 11.2 Environment variables

| Where  | Var                           | Purpose                                   |
|--------|-------------------------------|-------------------------------------------|
| portal | VITE_API_BASE_URL             | backend API base (blank in dev → proxy)   |
| portal | VITE_CLOUDINARY_CLOUD_NAME    | Cloudinary cloud                          |
| portal | VITE_CLOUDINARY_UPLOAD_PRESET | unsigned preset                           |
| server | CLOUDINARY_API_KEY / SECRET   | signed uploads / admin media (optional)   |
| server | ADMIN_JWT_SECRET              | admin token signing (or reuse JWT_SECRET) |
| server | LLM_* / EMBEDDINGS_*          | AI module (already present)               |
| server | SMTP_* / CASHFREE_*           | email / payments (already present)        |

### 11.3 Glossary

- **Catalog entity** — admin-managed list item shown in the app (provider, food item, doctor, temple...).
- **Offering** — a bookable service/package under a provider.
- **RAG** — Retrieval-Augmented Generation; the AI answers using admin-uploaded knowledge.
- **RBAC** — Role-Based Access Control.